Unix & Shell Programming

Module 2 – UNIX File Commands

Question & Answer Notes

BCA Semester 6 · MAKAUT

Q1. Commands to Create, Change, Move, and Copy Directories

mkdir – Make Directory (create a new directory)

```
mkdir mydir                 # create a single directory
mkdir dir1 dir2 dir3        # create multiple directories at once
mkdir -p a/b/c              # create nested directories (parent
   dirs created automatically)
```

cd – Change Directory

```
cd mydir                    # go into mydir
cd ..                       # go one level up (parent directory)
cd /home/user/documents     # go to an absolute path
cd ~                        # go to home directory
cd -                        # go to previous directory
```

mv – Move or Rename a Directory

```
mv oldname newname          # rename a directory
mv mydir /home/user/        # move mydir to another location
```

cp – Copy a Directory

```
cp -r source/ destination/    # -r means recursive (must use for
    directories)
cp -rp dir1 dir2              # -p preserves permissions and
   timestamps
```

Note: `-r` (recursive) is required with `cp` when copying directories. Without it, you get an error.

Q2. Commands to Create and Remove Files

Creating Files:

```
touch myfile.txt            # create an empty file (or update
   timestamps if it exists)
cat > myfile.txt            # create a file and type content (
   Ctrl+D to save)
echo "hello" > myfile.txt   # create a file with content
vi myfile.txt               # open vi editor to create/edit a
   file
```

Removing Files and Directories:

```
rm myfile.txt              # delete a file
rm -i myfile.txt           # ask for confirmation before
   deleting
rm -f myfile.txt           # force delete (no confirmation, no
   error if missing)
rm *.txt                   # delete all .txt files in current
   directory

rmdir mydir                # remove an EMPTY directory
rm -r mydir                # remove directory and all its
   contents (recursive)
rm -rf mydir               # force remove directory and contents
    (be careful!)
```

Warning: `rm -rf` is very dangerous. It deletes everything permanently with no undo. Always double-check the path before running it.

Q3. Listing Directory Information with Different Options

The **ls** command is used to list files and directories.

Basic Syntax: `ls [options] [path]`

Command	Description
`ls`	List files and folders in current directory
`ls -l`	Long listing – shows permissions, owner, size, date
`ls -a`	Show all files including hidden files (starting with .)
`ls -la`	Long listing + hidden files (combined)
`ls -lh`	Long listing with human-readable file sizes (KB, MB)
`ls -R`	Recursively list files in all subdirectories
`ls -t`	Sort by modification time (newest first)
`ls -r`	Reverse order of listing
`ls -i`	Show inode number of each file
`ls -s`	Show file size in blocks
`ls /etc`	List contents of a specific directory

Understanding `ls -l` output:

```
-rw-r--r-- 1 shadow users 1024 Jun 10 10:30 myfile.txt
```

Field	Example Value	Meaning
File type + permissions	`-rw-r--r--`	Type & read-/write/execute perms
Link count	`1`	Number of hard links
Owner	`shadow`	Who owns the file
Group	`users`	Owner's group
Size	`1024`	Size in bytes
Date & Time	`Jun 10 10:30`	Last modification time
Filename	`myfile.txt`	Name of the file

Q4. Different Types of Files in UNIX

In UNIX, *everything is a file*. UNIX supports 7 types of files, identified by the first character in `ls -l` output.

Type	Symbol	Description
Regular File	`-`	Ordinary file containing text, data, or program code. Example: `notes.txt`, `a.out`
Directory	`d`	A folder that contains other files and directories. Example: `/home/user`
Symbolic Link	`l`	A shortcut (pointer) to another file or directory. Example: `/bin/sh -> bash`
Block Device	`b`	Represents block storage devices like hard disks. Example: `/dev/sda`
Character Device	`c`	Represents character devices like keyboard, terminal. Example: `/dev/tty`
Named Pipe (FIFO)	`p`	Used for communication between processes (inter-process communication).
Socket	`s`	Used for network communication between processes.

```
ls -l /dev/sda     # b for block device
ls -l /dev/tty     # c for character device
ls -l /bin/sh      # l for symbolic link
```

Q5. Changing File Permissions – chmod

Every UNIX file has 3 sets of permissions for 3 types of users:

User Categories:

- **u** – User (owner of the file)

- **g** – Group (users in the same group)
- **o** – Others (everyone else)
- **a** – All (u + g + o)

Permission Types:

- **r** – Read (4) – can view file contents
- **w** – Write (2) – can modify file
- **x** – Execute (1) – can run as program

chmod – Change Mode

There are two ways to use `chmod`:

1. Symbolic Method:

```
chmod u+x file.sh          # give execute permission to owner
chmod g-w file.txt         # remove write permission from group
chmod o+r file.txt         # give read permission to others
chmod a+x script.sh        # give execute permission to everyone
chmod u+rwx,g+rx,o+r file  # set all at once
```

2. Numeric (Octal) Method:

Each permission has a number: r=4, w=2, x=1. Add them up for each group.

Permission	Owner	Group	Others	Meaning
`chmod 777`	rwx(7)	rwx(7)	rwx(7)	Everyone can do everything
`chmod 755`	rwx(7)	r-x(5)	r-x(5)	Owner full, others read+execute
`chmod 644`	rw-(6)	r–(4)	r–(4)	Owner read+write, others read only
`chmod 600`	rw-(6)	—(0)	—(0)	Only owner can read+write
`chmod 400`	r–(4)	—(0)	—(0)	Read-only for owner

```
chmod 755 myscript.sh      # common for executable scripts
chmod 644 myfile.txt       # common for regular text files
chmod 600 secret.txt       # private file, only owner reads it
```

Tip: To change permissions recursively on a directory: `chmod -R 755 mydir/`

Q6. touch Command – Access Time, Modification Time, Change Time

The **touch** command is used to:

1. Create a new empty file if it does not exist.

2. Update the timestamp of an existing file.

```
touch myfile.txt          # create empty file or update its
    timestamp
touch file1 file2 file3   # create/update multiple files at
    once
```

Three Types of Timestamps in UNIX:

Time Type	Short Form	When it Changes
Access Time	atime	When the file is *read* (e.g., `cat file`)
Modification Time	mtime	When the file *content is changed* (e.g., edited)
Change Time	ctime	When file *metadata changes* (permissions, owner, link count)

touch options:

```
touch -a file.txt         # update only access time (atime)
touch -m file.txt         # update only modification time (
    mtime)
touch -c file.txt         # do NOT create file if it doesn't
    exist
```

To view timestamps:

```
stat myfile.txt           # shows atime, mtime, and ctime
ls -l myfile.txt          # shows mtime only
ls -lu myfile.txt         # shows atime
ls -lc myfile.txt         # shows ctime
```

Q7. touch -d and touch -t Options

Both options allow you to **set a specific timestamp** on a file (instead of the current time).

touch -d (Date String):

`-d` accepts a human-readable date/time string.

```
touch -d "2024-01-15" file.txt
# Sets timestamp to January 15, 2024

touch -d "2024-06-10 09:30:00" file.txt
# Sets timestamp to June 10, 2024 at 9:30 AM
```

```
touch -d "yesterday" file.txt
# Sets timestamp to yesterday

touch -d "2 days ago" file.txt
# Sets timestamp to 2 days ago
```

touch -t (Timestamp):

`-t` accepts a timestamp in the format: `[[CC]YY]MMDDhhmm[.ss]`

```
touch -t 202401151030 file.txt
# Sets timestamp to 2024, Jan 15, 10:30 AM

touch -t 202406101500.30 file.txt
# Sets timestamp to 2024, Jun 10, 3:00:30 PM

touch -t 01151030 file.txt
# Assumes current year, Jan 15, 10:30 AM
```

Format breakdown for -t:

Part	Meaning	Example
`CC`	Century (optional)	20 (for 21st century)
`YY`	Year (optional)	24 (for 2024)
`MM`	Month	06 (June)
`DD`	Day	10
`hh`	Hour (24-hr)	15 (3 PM)
`mm`	Minutes	30
`.ss`	Seconds (optional)	.45

Difference: `-d` is easier and human-friendly. `-t` is strict numeric format. Use `-d` when you want to type a date naturally; use `-t` in scripts for precision.

Q8. Commands to Find Files and Directories

find – Search for files in a directory hierarchy

Basic syntax: `find [path] [options] [expression]`

```
find . -name "myfile.txt"          # find by name in current
   directory
find / -name "*.log"               # find all .log files from
   root
find . -type f                     # find only regular files
find . -type d                     # find only directories
find . -size +1M                   # find files larger than 1 MB
find . -size -100k                 # find files smaller than 100
   KB
```

```
find . -mtime -7                    # files modified in last 7
   days
find . -mtime +30                   # files not modified in 30+
   days
find . -user shadow                 # files owned by user 'shadow'
find . -perm 644                    # files with exactly 644
   permissions
find . -name "*.tmp" -delete        # find and delete all .tmp
   files
find . -name "*.sh" -exec chmod +x {} \;  # find and run a
   command on results
```

locate – Fast search using a database

```
locate myfile.txt           # quick search (uses pre-built index)
updatedb                    # update the locate database (as root
   )
```

which / whereis – Find command locations

```
which python3               # shows where python3 executable is
whereis ls                  # shows binary, source, and manual
   page locations
```

find vs locate: `find` searches in real-time (always up-to-date but slower). `locate` uses an indexed database (very fast but may be outdated).

Q9. Soft Links and Hard Links

A **link** is a reference (pointer) to a file. UNIX supports two types of links.

Hard Link:

A hard link is another *name* for the same file. Both the original and the hard link point to the same inode (actual data on disk).

```
ln original.txt hardlink.txt      # create a hard link
```

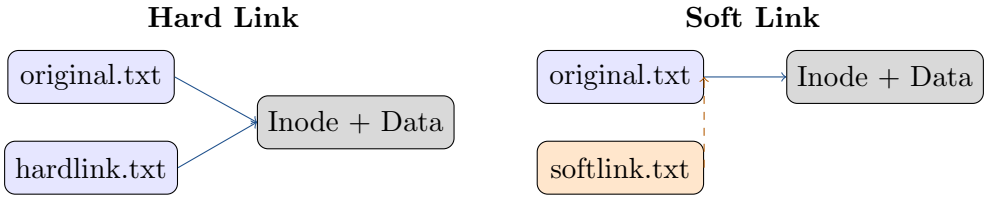
Soft Link (Symbolic Link):

A soft link (symlink) is a *shortcut* – it points to the path of another file, not directly to the inode.

```
ln -s original.txt softlink.txt  # create a soft link (-s for
   symbolic)
ln -s /home/user/docs ~/docs      # symlink to a directory
```

Comparison Table:

Feature	Hard Link	Soft Link
Points to	Same inode (data)	Path of the original file
Crosses file systems?	No	Yes
Works on directories?	No (generally)	Yes
If original deleted	Link still works (data remains)	Link breaks (dangling link)
Inode number	Same as original	Different (new inode)
ls -l symbol	No special marker	Shows `->` target
Size shown	Size of actual data	Size of path string



Q10. What is an Inode? Explain its Structure.

An **inode** (Index Node) is a data structure on disk that stores all information about a file *except its name and actual content.*

Every file in UNIX has exactly one inode. The directory entry maps a filename to an inode number.

Information stored in an Inode:

Field	Description
File type	Regular, directory, symlink, device, etc.
Permissions	Read, write, execute for owner, group, others
Owner (UID)	User ID of the file owner
Group (GID)	Group ID
File size	Size in bytes
Access time (atime)	Last time file was read
Modification time (mtime)	Last time file content was changed
Change time (ctime)	Last time inode metadata was changed
Link count	Number of hard links pointing to this inode
Data block pointers	Addresses of disk blocks storing actual data

What is NOT in an inode:

- Filename (stored in the directory, not the inode)
- File content (stored in data blocks on disk)

```
ls -i myfile.txt          # view inode number of a file
stat myfile.txt           # full inode information
df -i                     # show inode usage on file systems
```

How file access works: Filename → Directory lookup → Inode number → Inode → Data blocks → File content

Q11. UNIX File System Size

The UNIX file system has a concept of **blocks** – the smallest unit of disk storage. File size and disk usage are measured in terms of blocks.

Key concepts:

- **Block size:** Typically 512 bytes or 4096 bytes (4KB) depending on the file system.
- **File size vs Disk usage:** A file of 1 byte still occupies one full block on disk.
- **Disk usage** is always in whole blocks, so small files waste space (called internal fragmentation).

Commands to check file system size:

```
df -h                     # show disk space usage (human
   readable: KB, MB, GB)
df -i                     # show inode usage
du file.txt               # disk usage of a specific file (in
   blocks)
du -h file.txt            # disk usage in human-readable format
du -sh mydir/             # total size of a directory
du -ah                    # all files with sizes
ls -lh myfile.txt         # file size in human-readable format
```

Example `df -h` output:

```
Filesystem      Size  Used Avail Use%  Mounted on
/dev/sda1        50G   12G   36G  25%  /
tmpfs           2.0G  120M  1.9G   6%  /tmp
```

File System Size Components:

Component	Description
Boot block	First block, contains bootstrap code to boot the OS
Super block	Stores metadata about the file system (size, free blocks, inode count)
Inode table	Array of all inodes on the file system
Data blocks	Actual storage area where file contents are kept

Q12. Use of > and >> Operators

These are **output redirection** operators. By default, command output goes to the screen (stdout). These operators redirect it to a file instead.

> – Output Redirection (Overwrite):

Sends output to a file. If the file exists, it is *overwritten*. If it doesn't exist, it is created.

```
echo "Hello World" > greet.txt      # write to file (overwrite)
ls -l > filelist.txt                # save directory listing to
   file
date > today.txt                    # save current date to file
cat > newfile.txt                   # type content, Ctrl+D to
   finish
```

>> – Append Redirection:

Sends output to a file. Output is *added at the end* without overwriting existing content.

```
echo "Line 1" > log.txt             # creates file with "Line 1"
echo "Line 2" >> log.txt            # appends "Line 2"
echo "Line 3" >> log.txt            # appends "Line 3"
date >> log.txt                     # keep adding dates to the
   log
ls >> filelist.txt                  # add more files to list
```

Comparison:

Operator	If file exists	If file doesn't exist
>	Overwrites existing content	Creates a new file
>>	Appends to existing content	Creates a new file

Error Redirection:

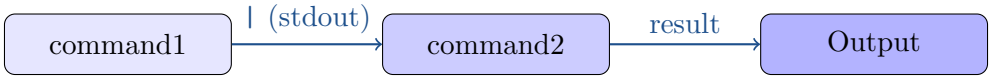
```
ls /invalid 2> error.txt            # redirect error (stderr) to
   file
ls /path > out.txt 2>&1             # redirect both stdout and
   stderr to file
```

Remember: `>` = Overwrite (dangerous if file has data). `>>` = Append (safe, adds to end).

Q13. What is a Pipe? Explain with Examples.

A **pipe** (`|`) is used to send the *output of one command as the input to another command*. It connects two commands together.

Syntax: `command1 | command2`



Examples:

```
ls -l | more                          # view long listing one page
    at a time
ls -l | less                          # scrollable view of
    directory listing

cat file.txt | grep "error"           # find lines with "error" in
    file
cat file.txt | wc -l                  # count number of lines in
    file

ps aux | grep firefox                 # find if firefox is running
ps aux | sort -k3 -r | head -10       # show top 10 CPU-using
    processes

ls | sort                             # list files in sorted order
ls | sort | uniq                      # sorted unique list

cat /etc/passwd | cut -d: -f1         # extract usernames from
    passwd file

who | wc -l                           # count how many users are
    logged in
```

Multiple Pipes (Pipeline):

You can chain many commands together:

```
cat file.txt | grep "import" | sort | uniq -c | sort -rn
# Find all "import" lines, sort, count unique, show most
    frequent first
```

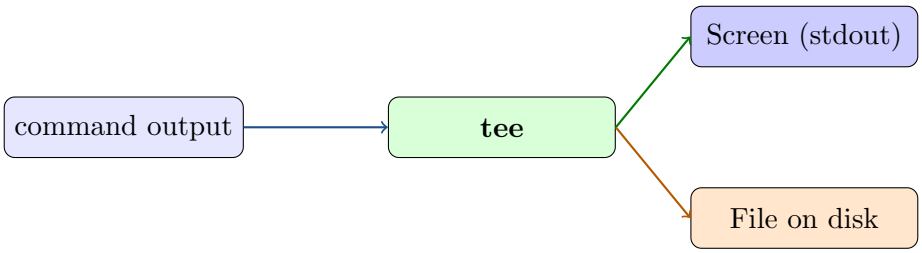
Key Point: Pipes work in memory – no temporary files are created. The data flows directly from one command to the next.

Q14. The tee Command

The **tee** command reads from standard input and writes to *both a file and the screen (stdout) at the same time.*

It is like a T-junction in a pipe – data flows to two destinations.

Syntax: `command | tee [options] filename`



Basic Examples:

```
ls -l | tee output.txt
# Displays ls -l on screen AND saves it to output.txt

date | tee today.txt
# Shows date on screen AND writes it to today.txt
```

tee with -a (append):

```
echo "New entry" | tee -a log.txt
# Appends to log.txt instead of overwriting it
```

tee with multiple files:

```
ls | tee file1.txt file2.txt
# Output goes to screen, file1.txt, AND file2.txt
```

tee in the middle of a pipeline:

```
cat file.txt | tee backup.txt | grep "error"
# Saves full content to backup.txt, then filters and shows only
    "error" lines
```

Command	Screen Output	File Output
`cmd > file`	No	Yes (overwrite)
`cmd >> file`	No	Yes (append)
`cmd | tee file`	Yes	Yes (overwrite)
`cmd | tee -a file`	Yes	Yes (append)

Use case: `tee` is very useful in scripts where you want to log output to a file while also seeing it live on the terminal.

— End of Module 2 Notes —